



Method Overriding :

When the function having same name and same argument And one version is present in parent class and another version is present in child class with the same signature.

Program: `#include <iostream>`
`using namespace std;`

Class Parent

```
{
    public:
        void show()
        {
            cout << "This is Parent class show() in";
        }
}
```

Class Child : public Parent

```
{
    public:
        void show() // overriding
        {
            cout << "This is child class show() in";
        }
}
```

int main()

```
{
    Child obj;
}
```

obj.show();

}

O/P :

This is child class show()

Here if we make parent class object and then call show() function → Then the parent class show function will execute

Important :

When we use objects directly (not pointers).

The function is called based on the object type.

If we can call different class function by making the objects of these class so then why we need to use base class pointer for that?

So making different class object for accessing the different functions of the inheritance class is not flexible and not a good practice.

As we can understand it by real-life :

Think of remote control :

Remote = parent pointer

TV/AC/FAN = child objects

Button = virtual function

You don't care what device is connected.



You press ONE button, correct device behavior happens

Without pointer :

We need different remote for every device

Secondary :

To implement runtime polymorphism in C++ we need base class pointer.

More perfectly :

As we know Runtime polymorphism is implemented through inheritance function and virtual function.

Runtime polymorphism in C++ is implemented using virtual functions and it works through base class pointer.